ГУАП

КАФЕДРА № 43

ОТЧЕТ
ЗАЩИЩЕН С ОЦЕНКОЙ

ПРЕПОДАВАТЕЛЬ

доцент		Попов А.А.
должность, уч. степень, звание	подпись, дата	инициалы, фамилия

ОТЧЕТ ПО ЛАБОРАТОРНОЙ РАБОТЕ №5

Автоматное программирование

по курсу: ПРОГРАММИРОВАНИЕ ВСТРОЕННЫХ ПРИЛОЖЕНИЙ

РАБОТУ ВЫПОЛНИЛ

СТУДЕНТ гр. №	4236		А. Панченко
		подпись, дата	инициалы, фамилия

Санкт-Петербург 2025

Цель работы:

Освоение принципов автоматного программирования как механизма организации управления периферийными подсистемами микроконтроллера. Изучение принципов работы с интерфейсами: универсальным асинхронным приёмопередатчиком UART; протоколом последовательной связи I 2C. Изучение механизмов прямого доступа к памяти (DMA).

Номер варианта с заданиями:

№	Описание задачи	Время одной итерации, сек
16	Замена a на bb. На ленте расположено слово из символов a, b и c, например abacbcab. Требуется заменить последний символ a на bb (для приведенного примера конечная запись на ленте будет иметь вид: abacbcbbb).	1
20	Удалить два последних a. На ленте расположено слово из букв a и b, например: baabba. Требуется удалить из слова две последние буквы a (для приведенного примера результат: babb).	1
24	Удаление парных скобок. На ленте расположена последовательность открывающих и закрывающих скобок, ограниченная пустыми символами, например)(()(()). Требуется удалить из последовательности все парные скобки, т.е. для приведенного примера результат будет иметь вид:)xxx((x.	1

Графы автоматов и их код:

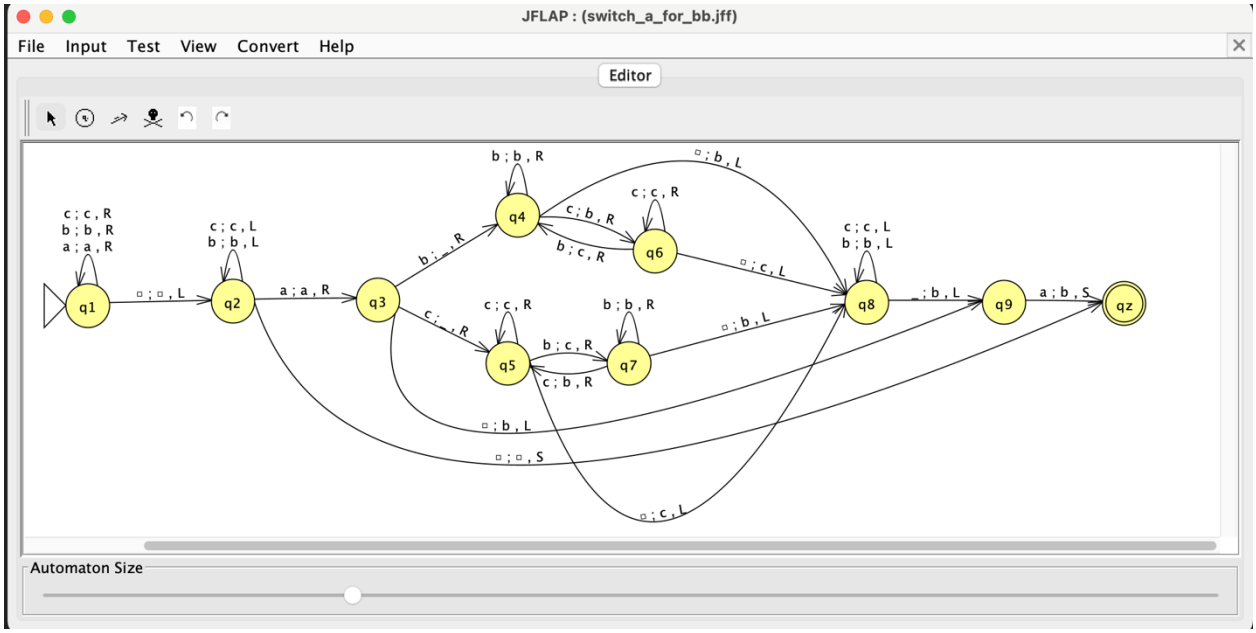


Рисунок 1 – Замена a на bb

```
/*fsm_tm3.c*/
#include "fsm_tm3.h"
#include "fsm_main.h"
#include "timers.h"
#include "messages.h"
#include "i2cFram.h"
#include "uart.h"
#include "main.h"

/*состояния автомата*/

#define MTCTRL_RXCMD     0 // ожидание команды запуска
#define MTCTRL_PROC      1 // работа MT
#define MTCTRL_RDFRAM    2 // чтение FRAM
#define MTCTRL_WRFRAM    3 // запись FRAM
#define MTCTRL_UARTTR    4 // передача UART

#define FSMST_Q0         0 // запуск MT
```

```
#define FSMST_Q1            1
#define FSMST_Q2            2
#define FSMST_Q3            3
#define FSMST_Q4            4
#define FSMST_Q5            5
#define FSMST_Q6            6
#define FSMST_Q7            7
#define FSMST_Q8            8
#define FSMST_Q9            9
#define FSMST_QZ            10 // останов МТ

static uint32_t curraddrtm=0; // текущее положение каретки
static uint32_t nextaddrtm=0; // следущее положение каретки
int ProcTm3(uint8_t rsymb, uint8_t* wrsymb, int* steps); // итерация автомата МТ

// переменная состояния автомата контроля МТ
static char mtstatectrl = MTCTRL_RXCMD;
// переменная состояния автомата МТ
static char fsmstate = FSMST_Q1;

int ProcessFsmTm3(void)
{
    static int steptm;
    static uint8_t rdsymbol;
    static uint8_t wrsymbol;
    static uint8_t aTrbuf[32];
    uint8_t* pRxbuf=0;
    int curstep=0;
    static int steps3 = -1;
```

```
switch (mtstatectrl)
{
	case MTCTRL_RXCMD: // ожидание и анализ команды
		if(GetMessage(MSG_TM3STRT,(void*)&pRxbuf))
		{
			pRxbuf[lenfsm]=0;
			nextaddrtm=-1;
			steptm=-1;

sscanf((char*)pRxbuf,"%x:%x",&nextaddrtm,&steptm);
			if(nextaddrtm>=0 && nextaddrtm<=0x7FF && steptm > 0)
			{// запуск MT
				mtstatectrl = MTCTRL_RDFRAM;
				fsmstate = FSMST_Q0;
				ResetTimer(TM1_TIMER);
			}
			else
			{// синтаксическая ошибка в команде
				strcpy((char*)aTrbuf,"Err> cmd error\n");
				mtstatectrl = MTCTRL_UARTTR;
				steptm = -1;
				steps3 = -1;
			}
		}
		break;
	case MTCTRL_PROC:
		curstep=ProcTm3(rdsymbol,&wrsymbol, &steps3);
		if(curstep==-1 || curstep == steptm)
		{// остановка по стоп(-1) или количеству шагов
```

```
                        sprintf((char*)aTrbuf,"TM3>stop        addr=%03X
sym='%c'\n",curraddrtm,wrsymbol);
                        steptm = -1;
                        steps3 = -1;
                }
                else
                        sprintf((char*)aTrbuf,"TM3>step=%d   addr=%03X
sym='%c'\n",steps3,curraddrtm,wrsymbol);
                if(wrsymbol == rdsymbol) // если символы совпадают то
запись пропускаем
                        mtstatectrl = MTCTRL_UARTTR;
                else
                        mtstatectrl = MTCTRL_WRFRAM;
                break;
        case MTCTRL_RDFRAM:
                curraddrtm=nextaddrtm; // переводим каретку
                pRxbuf = (uint8_t*)i2cFRAM_rd(curraddrtm,1);
                if( (uint32_t)pRxbuf > 0)
                {//читаем один байт под кареткой
                        rdsymbol=pRxbuf[0];
                        mtstatectrl = MTCTRL_PROC;
                }
                break;
        case MTCTRL_WRFRAM: //пишем один байт
                if((uint32_t)i2cFRAM_wr(curraddrtm,&wrsymbol,1) > 0)
                        mtstatectrl = MTCTRL_UARTTR;
                break;
        case MTCTRL_UARTTR:
                if(GetTimer(TM3_TIMER) > 100) // 1000 msec
                { // ожидаем время итерации
```

```
                        lenfsm = strlen((char*)aTrbuf);
                        if( uart_transmit(aTrbuf,lenfsm) == 0 )
                        { // передаём результат итерации в терминал
                                ResetTimer(TM3_TIMER);
                                if(steptm == -1)
                                        mtstatectrl = MTCTRL_RXCMD;
                                else
                                        mtstatectrl = MTCTRL_RDFRAM;
                        }
                }
                break;
        }
        return steps3;
}

int ProcTm3(uint8_t rsymb,uint8_t* wrsymb, int* steps)
{
        static int stepcnt=0;
        *wrsymb=rsymb;
        stepcnt++;
        switch (fsmstate)
        {
                case FSMST_Q0:
                        stepcnt=0;
                case FSMST_Q1:
                        if (rsymb == 'a' || rsymb == 'b' || rsymb == 'c')
                        {
                                nextaddrtm = curraddrtm + 1;
            fsmstate = FSMST_Q1;
                        }
```

```
        else if (rsymb == '_')
        {
                nextaddrtm = curraddrtm - 1;
                fsmstate = FSMST_Q2;
        }
        break;
case FSMST_Q2:
        if (rsymb == 'b' || rsymb == 'c')
        {
                nextaddrtm = curraddrtm - 1;
        }
        else if (rsymb == '_')
        {
                fsmstate = FSMST_QZ;
        }
        else if (rsymb == 'a')
        {
                nextaddrtm = curraddrtm + 1;
                fsmstate = FSMST_Q3;
        }
        break;
case FSMST_Q3:
        if (rsymb == 'b')
        {
                *wrsymb = '_';
                nextaddrtm = curraddrtm + 1;
                fsmstate = FSMST_Q4;
        }
        else if (rsymb == 'c')
        {
```

```
                *wrsymb = '_';
                nextaddrtm = curraddrtm + 1;
                fsmstate = FSMST_Q5;
        }
        else if (rsymb == '_')
        {
                *wrsymb = 'b';
                nextaddrtm = curraddrtm - 1;
                fsmstate = FSMST_Q9;
        }
        break;
case FSMST_Q4:
        if (rsymb == 'b')
        {
                nextaddrtm = curraddrtm + 1;
        }
        else if (rsymb == 'c')
        {
                *wrsymb = 'b';
                nextaddrtm = curraddrtm + 1;
                fsmstate = FSMST_Q6;
        }
        else if (rsymb == '_')
        {
                *wrsymb = 'b';
                nextaddrtm = curraddrtm - 1;
                fsmstate = FSMST_Q8;
        }
        break;
case FSMST_Q5:
```

```
        if (rsymb == 'c')
        {
                nextaddrtm = curraddrtm + 1;
        }
        else if (rsymb == 'b')
        {
                *wrsymb = 'c';
                nextaddrtm = curraddrtm + 1;
                fsmstate = FSMST_Q7;
        }
        else if (rsymb == '_')
        {
                *wrsymb = 'c';
                nextaddrtm = curraddrtm - 1;
                fsmstate = FSMST_Q8;
        }
        break;
case FSMST_Q6:
        if (rsymb == 'c')
        {
                nextaddrtm = curraddrtm + 1;
        }
        else if (rsymb == '_')
        {
                *wrsymb = 'c';
                nextaddrtm = curraddrtm - 1;
                fsmstate = FSMST_Q8;
        }
        else if (rsymb == 'b')
        {
```

```
                *wrsymb = 'c';
                nextaddrtm = curraddrtm + 1;
                fsmstate = FSMST_Q4;
        }
        break;
case FSMST_Q7:
        if (rsymb == 'b')
        {
                nextaddrtm = curraddrtm + 1;
        }
        else if (rsymb == '_')
        {
                *wrsymb = 'b';
                nextaddrtm = curraddrtm - 1;
                fsmstate = FSMST_Q8;
        }
        else if (rsymb == 'c')
        {
                *wrsymb = 'b';
                nextaddrtm = curraddrtm + 1;
                fsmstate = FSMST_Q5;
        }
        break;
case FSMST_Q8:
        if (rsymb == 'c' || rsymb == 'b')
        {
                nextaddrtm = curraddrtm - 1;
        }
        else if (rsymb == '_')
        {
```

```
                        *wrsymb = 'b';
                        nextaddrtm = curraddrtm - 1;
                        fsmstate = FSMST_Q9;
                }
                break;
        case FSMST_Q9:
                if (rsymb == 'a')
                {
                        *wrsymb = 'b';
                        fsmstate = FSMST_QZ;
                }
                break;
        case FSMST_QZ:
                stepcnt = -1;
                break;
    }
    *steps = stepcnt;
    return stepcnt;
}
```

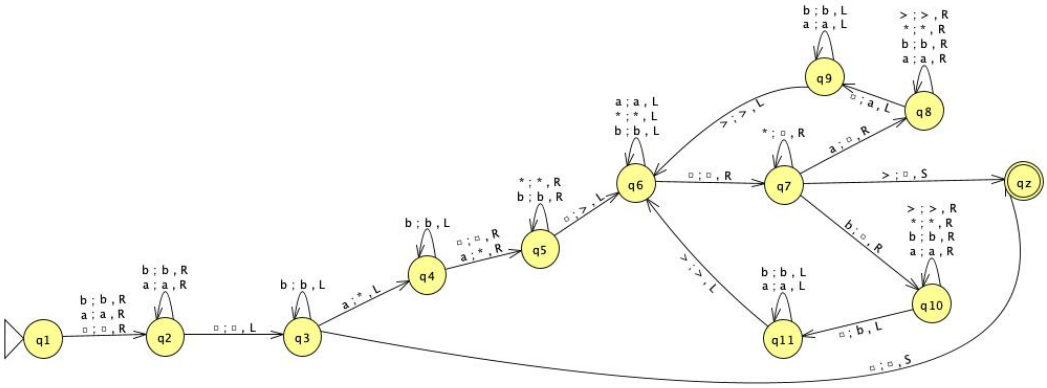


Рисунок 2 – Удалить два последних a

```
/*fsm_tm1.c*/
#include "fsm_tm1.h"
#include "fsm_main.h"
#include "timers.h"
#include "messages.h"
#include "i2cFram.h"
#include "uart.h"
#include "main.h"

/*состояния автомата*/

#define MTCTRL_RXCMD    0 // ожидание команды запуска
#define MTCTRL_PROC             1 // работа МТ
#define MTCTRL_RDFRAM  2 // чтение FRAM
#define MTCTRL_WRFRAM  3 // запись FRAM
#define MTCTRL_UARTTR   4 // передача UART

#define FSMST_Q0            0 // запуск МТ
#define FSMST_Q1            1
#define FSMST_Q2            2
#define FSMST_Q3            3
#define FSMST_Q4            4
#define FSMST_Q5            5
#define FSMST_Q6            6
#define FSMST_Q7            7
#define FSMST_Q8            8
#define FSMST_Q9            9
#define FSMST_Q10           10
#define FSMST_Q11           11
#define FSMST_QZ            12 // останов МТ
```

```
static uint32_t curraddrtm=0; // текущее положение каретки
static uint32_t nextaddrtm=0; // следущее положение каретки
int ProcTm1(uint8_t rsymb, uint8_t* wrsymb, int* steps); // итерация автомата МТ

// переменная состояния автомата контроля МТ
static char mtstatectrl = MTCTRL_RXCMD;
// переменная состояния автомата МТ
static char fsmstate = FSMST_Q1;

int ProcessFsmTm1(void)
{
	static int steptm;
	static uint8_t rdsymbol;
	static uint8_t wrsymbol;
	static uint8_t aTrbuf[32];
	uint8_t* pRxbuf=0;
	int curstep=0;
	static int steps1 = -1;
	switch (mtstatectrl)
	{
		case MTCTRL_RXCMD: // ожидание и анализ команды
			if(GetMessage(MSG_TM1STRT,(void*)&pRxbuf))
			{
				pRxbuf[lenfsm]=0;
				nextaddrtm=-1;
				steptm=-1;

sscanf((char*)pRxbuf,"%x:%x",&nextaddrtm,&steptm);
```

```
                    if(nextaddrtm>=0   &&   nextaddrtm<=0x7FF   &&
steptm > 0)
                    {// запуск MT
                        mtstatectrl = MTCTRL_RDFRAM;
                        fsmstate = FSMST_Q0;
                        ResetTimer(TM1_TIMER);
                    }
                    else
                    {// синтаксическая ошибка в команде
                        strcpy((char*)aTrbuf,"Err> cmd error\n");
                        mtstatectrl = MTCTRL_UARTTR;
                        steptm = -1;
                        steps1 = -1;
                    }
                }
                break;
            case MTCTRL_PROC:
        curstep=ProcTm1(rdsymbol,&wrsymbol, &steps1);
                if(curstep==-1 || curstep == steptm)
                {// остановка по стоп(-1) или количеству шагов
                    sprintf((char*)aTrbuf,"TM1>stop        addr=%03X
sym='%c'\n",curraddrtm,wrsymbol);
                    steptm = -1;
                    steps1 = -1;
                }
                else
                    sprintf((char*)aTrbuf,"TM1>step=%d   addr=%03X
sym='%c'\n",steps1,curraddrtm,wrsymbol);
                if(wrsymbol == rdsymbol) // если символы совпадают то
запись пропускаем
```

```
                mtstatectrl = MTCTRL_UARTTR;
        else
                mtstatectrl = MTCTRL_WRFRAM;
        break;
case MTCTRL_RDFRAM:
        curraddrtm=nextaddrtm; // переводим каретку
        pRxbuf = (uint8_t*)i2cFRAM_rd(curraddrtm,1);
        if( (uint32_t)pRxbuf > 0)
        {//читаем один байт под кареткой
                rdsymbol=pRxbuf[0];
                mtstatectrl = MTCTRL_PROC;
        }
        break;
case MTCTRL_WRFRAM: //пишем один байт
        if((uint32_t)i2cFRAM_wr(curraddrtm,&wrsymbol,1) > 0)
                mtstatectrl = MTCTRL_UARTTR;
        break;
case MTCTRL_UARTTR:
        if(GetTimer(TM1_TIMER) > 1000) // 1000 msec
        { // ожидаем время итерации
                lenfsm = strlen((char*)aTrbuf);
                if( uart_transmit(aTrbuf,lenfsm) == 0 )
                { // передаём результат итерации в терминал
                        ResetTimer(TM1_TIMER);
                        if(steptm == -1)
                                mtstatectrl = MTCTRL_RXCMD;
                        else
                                mtstatectrl = MTCTRL_RDFRAM;
                }
        }
```

```
                        break;
        }
    return steps1;
}

int ProcTm1(uint8_t rsymb,uint8_t* wrsymb, int* steps)
{
        static int stepcnt=0;
        *wrsymb=rsymb;
        stepcnt++;
        switch (fsmstate)
        {
                case FSMST_Q0:
                        stepcnt=0;
                case FSMST_Q1: // q1
                        if(rsymb == '_' || rsymb == 'a' || rsymb == 'b') {
                                // _,_,R
            // a,a,R
            // b,b,R
                                nextaddrtm = curraddrtm + 1;
                                fsmstate = FSMST_Q2;
                        }
                        break;
                case FSMST_Q2: // q2
                        if(rsymb == 'a' || rsymb == 'b') {
                                // a,a,R
            // b,b,R
                                nextaddrtm = curraddrtm + 1;
                        } else if(rsymb == '_') {
            // _,_,L
```

```
                nextaddrtm = curraddrtm - 1;
                fsmstate = FSMST_Q3;
            }
            break;
        case FSMST_Q3: // q3
            if(rsymb == 'b') {
                nextaddrtm = curraddrtm - 1;
            } else if(rsymb == 'a') {
                *wrsymb = '*'; // a,*,L
                nextaddrtm = curraddrtm - 1;
                fsmstate = FSMST_Q4;
            } else if(rsymb == '_') {
                // _,_,S
                fsmstate = FSMST_QZ;
            }
            break;
        case FSMST_Q4: // q4
            if(rsymb == 'b') {
                nextaddrtm = curraddrtm - 1;
            } else if(rsymb == 'a') {
                *wrsymb = '*'; // a,*,R
                nextaddrtm = curraddrtm + 1;
                fsmstate = FSMST_Q5;
            } else if(rsymb == '_') {
// _,_,R
                nextaddrtm = curraddrtm + 1;
                fsmstate = FSMST_Q5;
            }
            break;
        case FSMST_Q5: // q5
```

```
            if(rsymb == 'b' || rsymb == '*') {
                // b,b,R
                nextaddrtm = curraddrtm + 1;
            } else if(rsymb == '_') {
                *wrsymb = '>'; // _,>,L
                nextaddrtm = curraddrtm - 1;
                fsmstate = FSMST_Q6;
            }
            break;
    case FSMST_Q6: // q6
            if(rsymb == 'a' || rsymb == 'b' || rsymb == '*') {
                // a,a,L
// b,b,L
// *,*,L
                nextaddrtm = curraddrtm - 1;
            } else if(rsymb == '_') {
                // _,_,R
                nextaddrtm = curraddrtm + 1;
fsmstate = FSMST_Q7;
            }
            break;
    case FSMST_Q7: // q7
            if(rsymb == '*') {
                *wrsymb = '_'; // *,_,R
nextaddrtm = curraddrtm + 1;
            } else if(rsymb == 'a') {
                *wrsymb = '_'; // a,_,R
                nextaddrtm = curraddrtm + 1;
                fsmstate = FSMST_Q8;
            } else if(rsymb == 'b') {
```

```
                        *wrsymb = '_'; // b,_,R
                        nextaddrtm = curraddrtm + 1;
                        fsmstate = FSMST_Q10;
                } else if(rsymb == '>') {
                        *wrsymb = '_'; // >,_,S
                        fsmstate = FSMST_QZ;
                }
                break;
        case FSMST_Q8: // q8
                if(rsymb == 'a' || rsymb == 'b' || rsymb == '*' || rsymb ==
'>') {
                        // a,a,R
                // b,b,R
                // *,*,R
                // >,>,R
                        nextaddrtm = curraddrtm + 1;
                } else if(rsymb == '_') {
                        *wrsymb = 'a'; // _,a,L
                        nextaddrtm = curraddrtm - 1;
                        fsmstate = FSMST_Q9;
                }
                break;
        case FSMST_Q9: // q9
                if(rsymb == 'a' || rsymb == 'b') {
                        // a,a,L
                // b,b,L
                        nextaddrtm = curraddrtm - 1;
                } else if(rsymb == '>') {
                        // >,>,L
                        nextaddrtm = curraddrtm - 1;
```

```
                        fsmstate = FSMST_Q6;
                }
                break;
            case FSMST_Q10: // q10
                if(rsymb == 'a' || rsymb == 'b' || rsymb == '*' || rsymb ==
'>') {
                        // a,a,R
                // b,b,R
                // *,*,R
                // >,>,R
                        nextaddrtm = curraddrtm + 1;
                } else if(rsymb == '_') {
                        *wrsymb = 'b'; // _,b,L
                        nextaddrtm = curraddrtm - 1;
                        fsmstate = FSMST_Q11;
                }
                break;
            case FSMST_Q11: // q11
                if(rsymb == 'a' || rsymb == 'b') {
                        // a,a,L
                // b,b,L
                        nextaddrtm = curraddrtm - 1;
                } else if(rsymb == '>') {
                        // >,>,L
                        nextaddrtm = curraddrtm - 1;
                        fsmstate = FSMST_Q6;
                }
                break;
            case FSMST_QZ: // qz (final state)
                stepcnt = -1; // Halt
```

```
                    break;
            }
    *steps = stepcnt;
            return stepcnt;
}
```

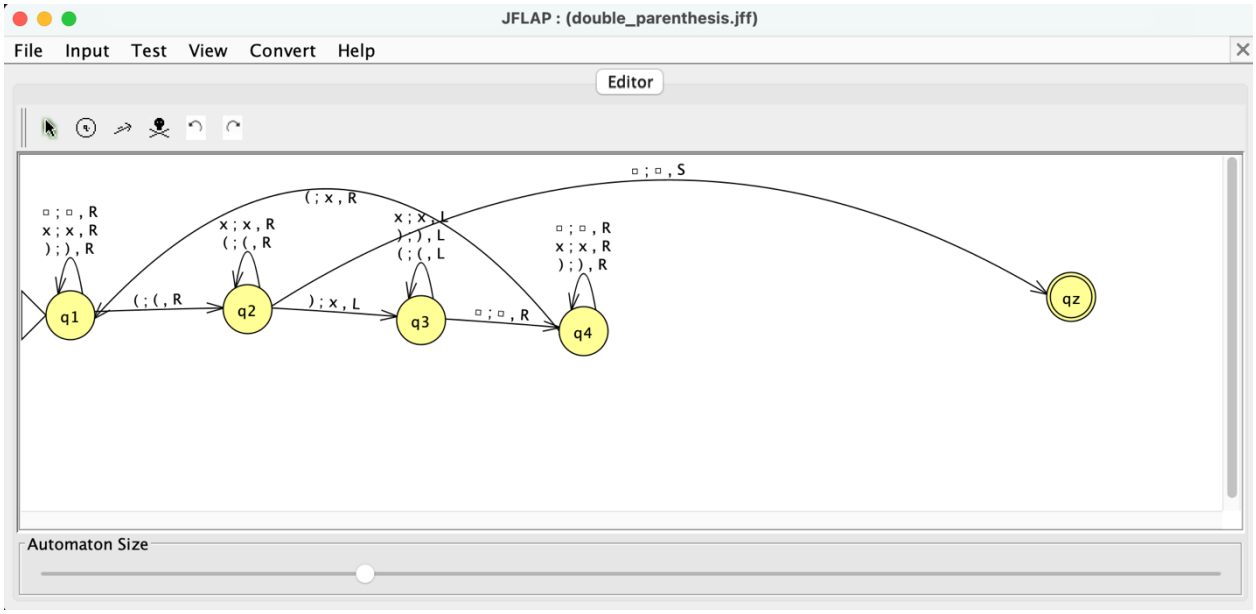


Рисунок 3 – Удаление парных скобок

```
/*fsm_tm2.c*/
#include "fsm_tm2.h"
#include "fsm_main.h"
#include "timers.h"
#include "messages.h"
#include "i2cFram.h"
#include "uart.h"
#include "main.h"

/*состояния автомата*/

#define MTCTRL_RXCMD     0 // ожидание команды запуска
#define MTCTRL_PROC              1 // работа МТ
```

```
#define MTCTRL_RDFRAM  2 // чтение FRAM
#define MTCTRL_WRFRAM  3 // запись FRAM
#define MTCTRL_UARTTR  4 // передача UART

#define FSMST_Q0       0 // запуск МТ
#define FSMST_Q1       1
#define FSMST_Q2       2
#define FSMST_Q3       3
#define FSMST_Q4       4
#define FSMST_QZ       5 // останов МТ

static uint32_t curraddrtm=0; // текущее положение каретки
static uint32_t nextaddrtm=0; // следущее положение каретки
int ProcTm2(uint8_t rsymb, uint8_t* wrsymb, int* steps); // итерация автомата МТ

// переменная состояния автомата контроля МТ
static char mtstatectrl = MTCTRL_RXCMD;
// переменная состояния автомата МТ
static char fsmstate = FSMST_Q1;

int ProcessFsmTm2(void)
{
    static int steptm;
    static uint8_t rdsymbol;
    static uint8_t wrsymbol;
    static uint8_t aTrbuf[32];
    uint8_t* pRxbuf=0;
    int curstep=0;
    static int steps2 = -1;
```

```
switch (mtstatectrl)
{
	case MTCTRL_RXCMD: // ожидание и анализ команды
		if(GetMessage(MSG_TM2STRT,(void*)&pRxbuf))
		{
			pRxbuf[lenfsm]=0;
			nextaddrtm=-1;
			steptm=-1;

sscanf((char*)pRxbuf,"%x:%x",&nextaddrtm,&steptm);
			if(nextaddrtm>=0 && nextaddrtm<=0x7FF && steptm > 0)
			{// запуск MT
				mtstatectrl = MTCTRL_RDFRAM;
				fsmstate = FSMST_Q0;
				ResetTimer(TM2_TIMER);
			}
			else
			{// синтаксическая ошибка в команде
				strcpy((char*)aTrbuf,"Err> cmd error\n");
				mtstatectrl = MTCTRL_UARTTR;
				steptm = -1;
				steps2 = -1;
			}
		}
		break;
	case MTCTRL_PROC:
		curstep=ProcTm2(rdsymbol,&wrsymbol, &steps2);
		if(curstep==-1 || curstep == steptm)
		{// остановка по стоп(-1) или количеству шагов
```

```
                        sprintf((char*)aTrbuf,"TM2>stop        addr=%03X
sym='%c'\n",curraddrtm,wrsymbol);
                        steptm = -1;
                        steps2 = -1;
                }
                else
                        sprintf((char*)aTrbuf,"TM2>step=%d   addr=%03X
sym='%c'\n",steps2,curraddrtm,wrsymbol);
                if(wrsymbol == rdsymbol) // если символы совпадают то
запись пропускаем
                        mtstatectrl = MTCTRL_UARTTR;
                else
                        mtstatectrl = MTCTRL_WRFRAM;
                break;
        case MTCTRL_RDFRAM:
                curraddrtm=nextaddrtm; // переводим каретку
                pRxbuf = (uint8_t*)i2cFRAM_rd(curraddrtm,1);
                if( (uint32_t)pRxbuf > 0)
                {//читаем один байт под кареткой
                        rdsymbol=pRxbuf[0];
                        mtstatectrl = MTCTRL_PROC;
                }
                break;
        case MTCTRL_WRFRAM: //пишем один байт
                if((uint32_t)i2cFRAM_wr(curraddrtm,&wrsymbol,1) > 0)
                        mtstatectrl = MTCTRL_UARTTR;
                break;
        case MTCTRL_UARTTR:
                if(GetTimer(TM2_TIMER) > 1000) // 1000 msec
                { // ожидаем время итерации
```

```
                        lenfsm = strlen((char*)aTrbuf);
                        if( uart_transmit(aTrbuf,lenfsm) == 0 )
                        { // передаём результат итерации в терминал
                                ResetTimer(TM2_TIMER);
                                if(steptm == -1)
                                        mtstatectrl = MTCTRL_RXCMD;
                                else
                                        mtstatectrl = MTCTRL_RDFRAM;
                        }
                }
                break;
        }
    return steps2;
}

int ProcTm2(uint8_t rsymb,uint8_t* wrsymb, int* steps)
{
        static int stepcnt=0;
        *wrsymb=rsymb;
        stepcnt++;
        switch (fsmstate)
        {
                case FSMST_Q0:
                        stepcnt=0;
                case FSMST_Q1:
                        if (rsymb == '(')
                        {
                                nextaddrtm = curraddrtm + 1;
                                fsmstate = FSMST_Q2;
                        }
```

```
                else if (rsymb == '_' || rsymb == 'x' || rsymb == ')')
{
                        nextaddrtm = curraddrtm + 1;
    fsmstate = FSMST_Q1;
}
                break;
        case FSMST_Q2:
                if (rsymb == ')')
                {
                        *wrsymb = 'x';
                        nextaddrtm = curraddrtm - 1;
                        fsmstate = FSMST_Q3;
                }
                else if (rsymb == 'x' || rsymb == '(')
                        nextaddrtm = curraddrtm + 1;
else if (rsymb == '_')
                        fsmstate = FSMST_QZ;
                break;
        case FSMST_Q3:
                if (rsymb == '_')
                {
                        nextaddrtm = curraddrtm + 1;
                        fsmstate = FSMST_Q4;
                }
                else if (rsymb == 'x' || rsymb == '(' || rsymb == ')')
                        nextaddrtm = curraddrtm - 1;
                break;
        case FSMST_Q4:
                if (rsymb == '(')
                {
```

```
                        *wrsymb = 'x';
                        nextaddrtm = curraddrtm + 1;
                        fsmstate = FSMST_Q1;
                }
                else if (rsymb == '_' || rsymb == 'x' || rsymb == ')')
                        nextaddrtm = curraddrtm + 1;
                break;
            case FSMST_QZ:
                stepcnt = -1;
                break;
        }
    *steps = stepcnt;
        return stepcnt;
}
```

Скриншоты работы с осциллографом:

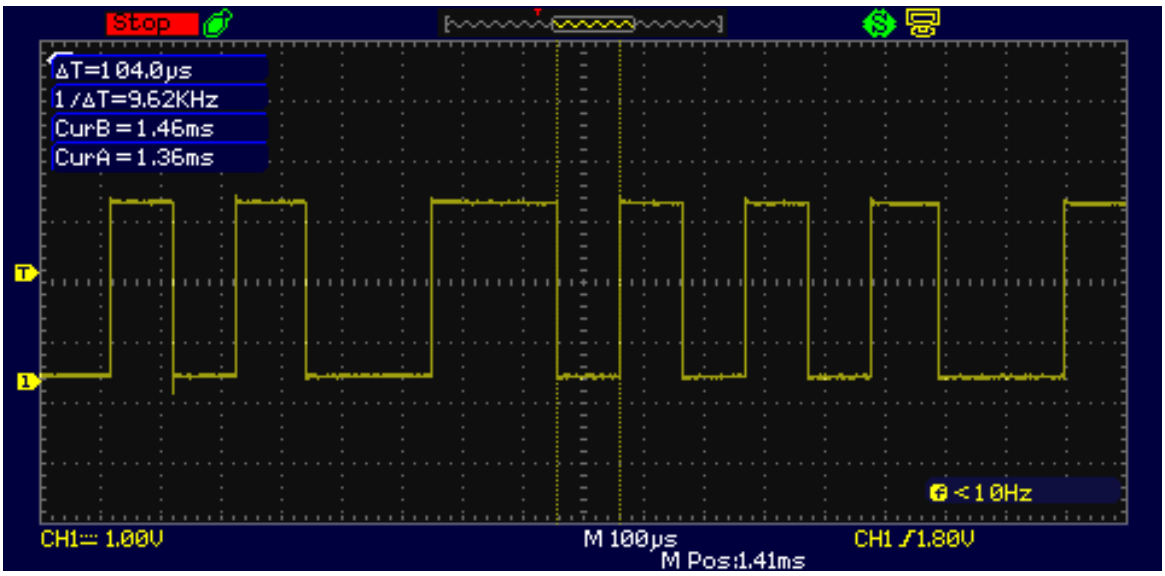
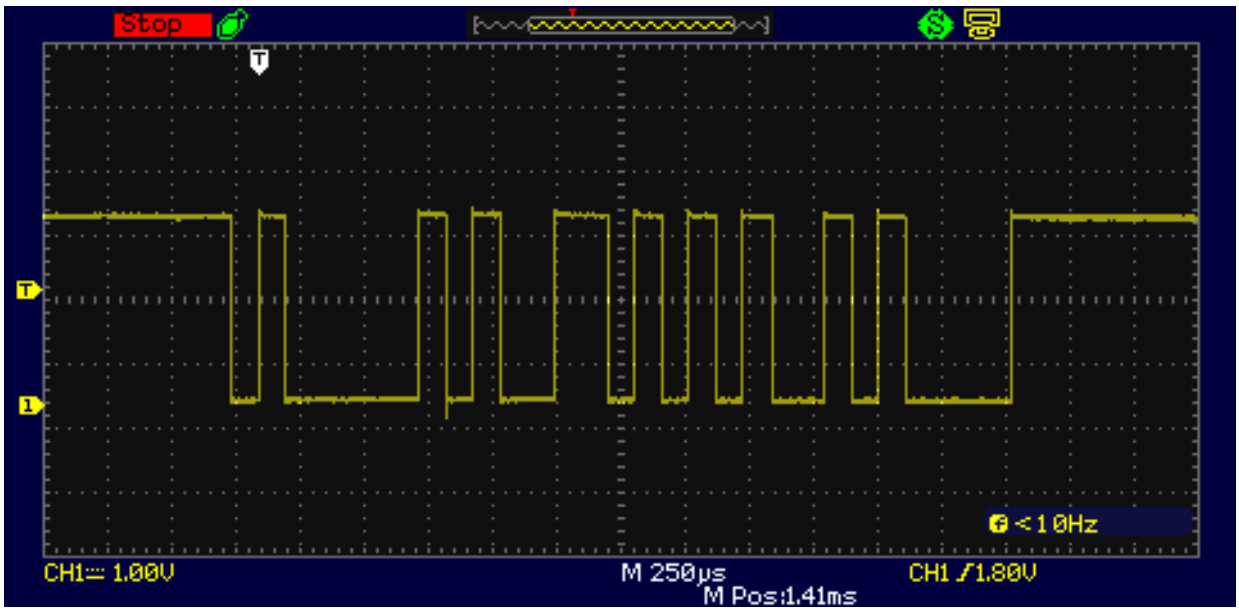
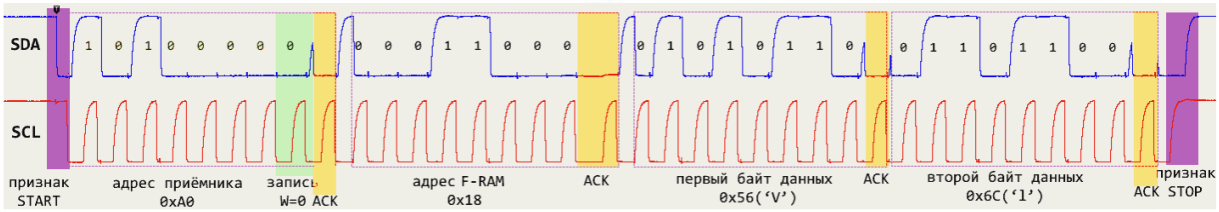


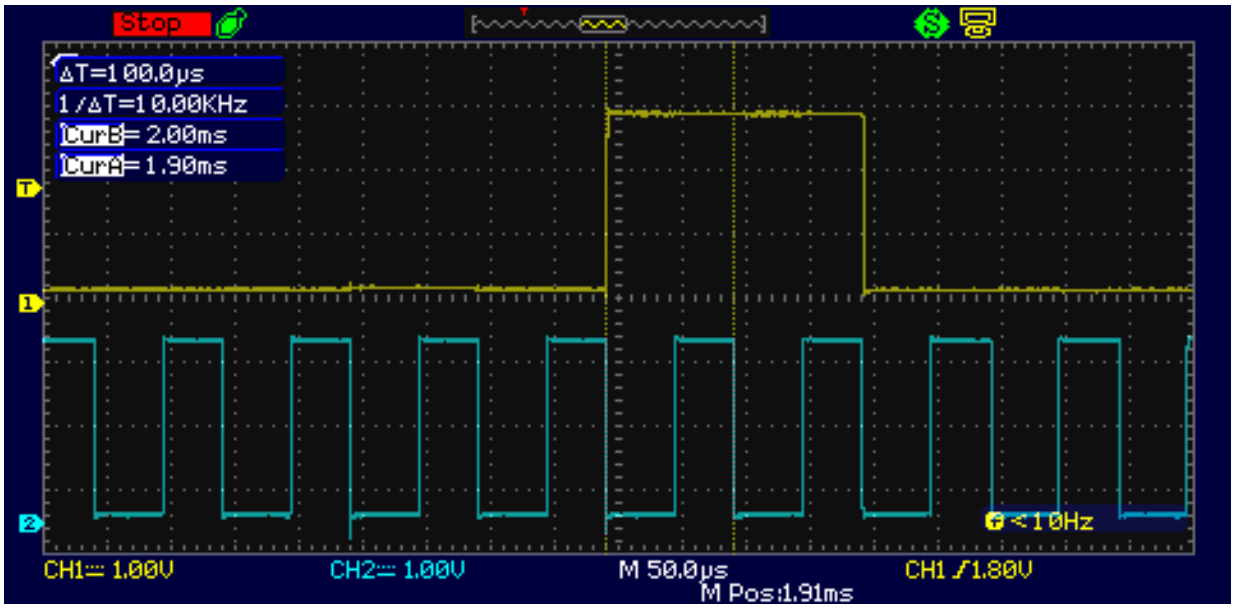
Рисунок 4 – Буквы AV инициалы курсоры передача по UART



AV = 10000011010110 = 20D6

Рисунок 5 – буквы AV инициалы передача по UART





ab = 11000011100010 = 30E2

Рисунок 6 – Два канала ab курсоры период

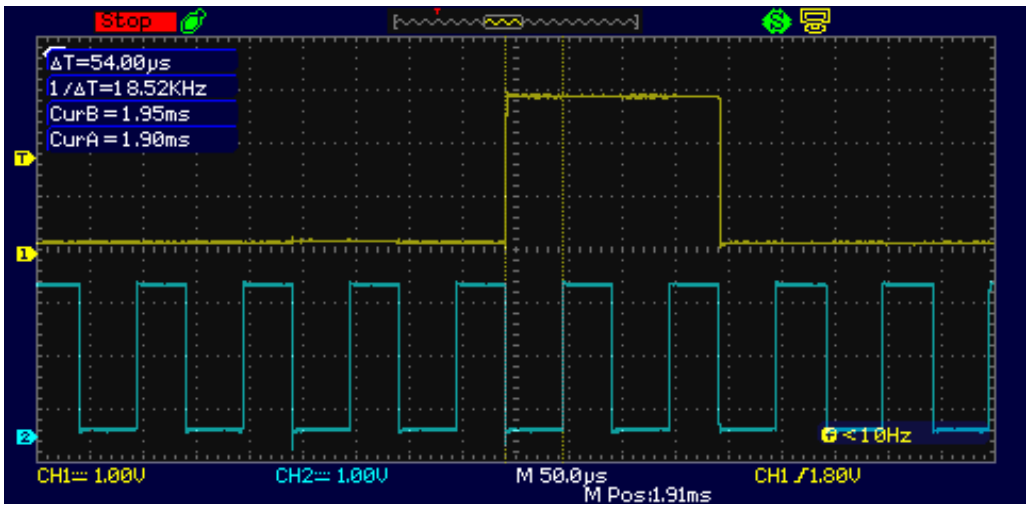
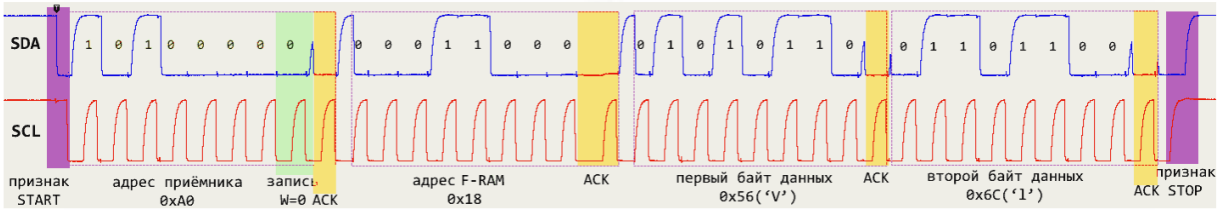


Рисунок 7 – Два канала ab курсоры полупериод



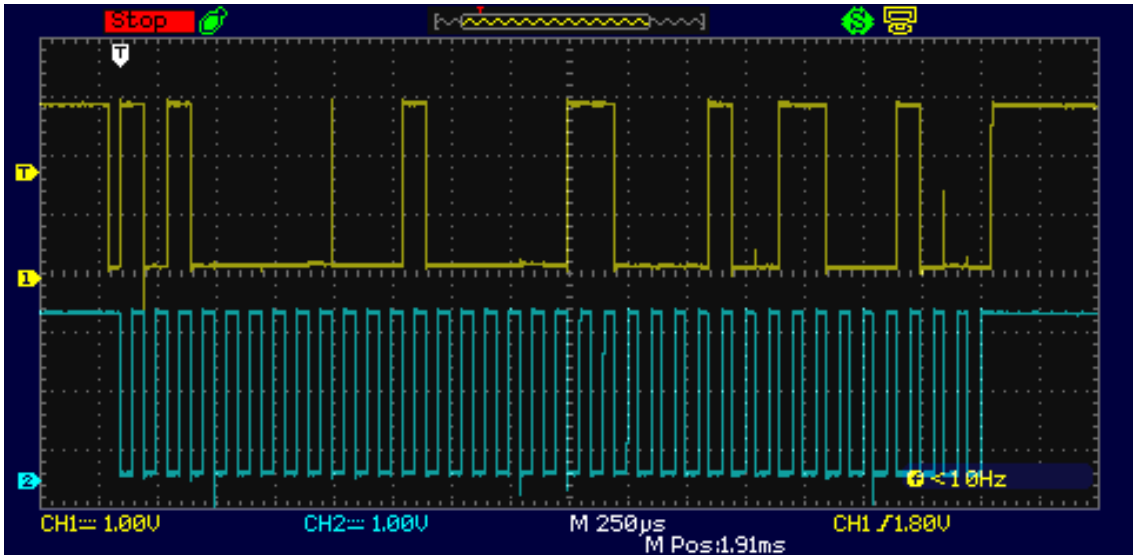


Рисунок 8 – Два канала ab полный график

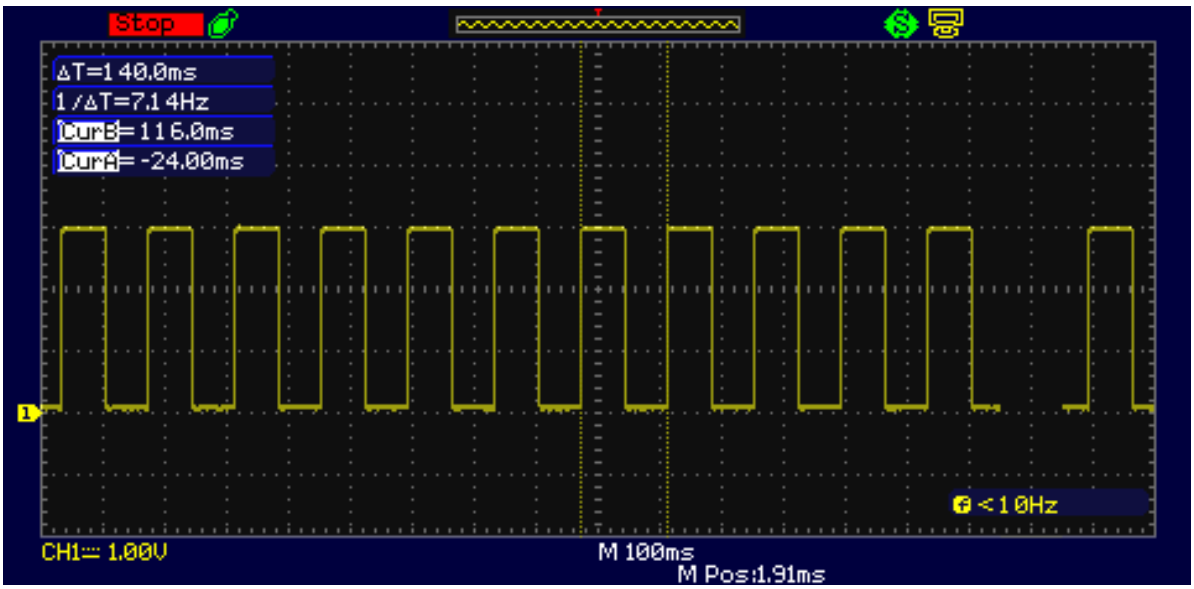


Рисунок 9 – Мерцание светодиода

Вывод:

В ходе работы были освоены принципы автоматного программирования для управления периферийными подсистемами микроконтроллера, изучены особенности работы с интерфейсами UART и I^2C, а также механизмы прямого доступа к памяти (DMA), что позволило углубить понимание организации эффективного взаимодействия между компонентами микроконтроллерных систем и оптимизировать обработку данных.